ENGINEERING ESSENTIALS FAANG INTERVIEW MASTER SERIES

Algorithms & Complexity

Big-O, sorting & searching, graph algorithms, DP families and greedy — with code and trade-offs.

Big-O | Sorting | Binary Search | Dijkstra | Topological Sort | Dynamic Programming | Greedy | Bit Tricks

VOLUME 14
THE DSA ALGORITHMS ISSUE

COMPREHENSIVE
DEEP DIVE + REVISION

LEVEL
SWE → STAFF

IN THIS ISSUE

Contents

01 Overview

02 Complexity Analysis

03 Algorithms

04 Algorithm Selection Cheat Table

05 Real-Life Analogies

06 Common Interview Questions

07 Typical Mistakes Candidates Make

08 Revision Cheat Sheet

SECTION 01

01 Overview

Algorithmic thinking is the foundation of every coding interview. You are not just expected to solve problems — you are expected to **reason about efficiency**, identify optimal approaches, and communicate trade-offs clearly. This file covers the complete algorithmic toolkit needed for top-tier software engineering interviews.

What interviewers evaluate: 1. Can you identify what kind of problem this is? (sorting, graph, DP, greedy...) 2. Do you know the right algorithm and its complexity? 3. Can you implement it correctly under pressure? 4. Can you articulate trade-offs and edge cases?

Core skill loop: Recognize → Design → Analyze → Implement → Verify → Optimize

SECTION 02

02 Complexity Analysis

Big-O / Big-Theta / Big-Omega

Complexity notation describes **how resource usage scales with input size**, not actual runtime.

Notation	Name	Meaning
O(f(n))	Big-O (upper bound)	Algorithm runs in **at most** f(n) steps (worst case)
Ω(f(n))	Big-Omega (lower bound)	Algorithm runs in **at least** f(n) steps (best case)
Θ(f(n))	Big-Theta (tight bound)	Algorithm runs in **exactly** f(n) steps (avg case matches both)

Formal definition — Big-O: $T(n) = O(f(n))$ if $\exists$ constants $c > 0$ and n_0 such that $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$.

Complexity hierarchy (slowest to fastest growth):

DIAGRAM

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Dropping constants and lower-order terms: - $O(3n^2 + 5n + 12) \rightarrow O(n^2)$ - $O(100) \rightarrow O(1)$ - $O(n + n \log n) \rightarrow O(n \log n)$

Multi-variable complexity: When input has multiple dimensions, keep all: - $O(V + E)$ for graph algorithms (V = vertices, E = edges) - $O(m \cdot n)$ for 2D DP problems

Amortized Analysis

Amortized analysis gives the **average cost per operation over a sequence**, even if individual operations are expensive.

Example — Dynamic Array (Python list append): - Most appends: O(1) - Occasional doubling: O(n) - Amortized cost per append: O(1) — because doubling happens exponentially rarely

Three methods: | Method | Idea | When to use | |--------|------|-------------| | Aggregate | Total cost / n operations | Simplest; works when all operations have same amortized cost | | Accounting | Prepay credits for future expensive ops | When operations have different costs | | Potential | Φ(state) function tracks "stored energy" | Most general; used for complex data structures |

Key amortized results: - Dynamic array append: O(1) amortized - Union-Find with union-by-rank + path compression: $O(\alpha(n))$ amortized $\approx$ O(1) - Splay tree operations: O(log n) amortized - Fibonacci heap decrease-key: O(1) amortized

Computing Time & Space Complexity

Time complexity rules: 1. **Sequential statements:** Add costs — O(A) + O(B) = O(max(A, B)) 2. **Nested loops:** Multiply — outer × inner 3. **Function calls:** Substitute the called function's complexity 4. **Recursion:** Use recurrence relations or Master Theorem

PYTHON

```python
# O(n) — single loop
for i in range(n):
    do_constant_work()

# O(n²) — nested loops
for i in range(n):
    for j in range(n):
        do_constant_work()

# O(n log n) — loop + halving inner
for i in range(n):
    j = n
    while j > 1:
        j //= 2   # log n iterations

# O(log n) — binary search
lo, hi = 0, n
while lo < hi:
    mid = (lo + hi) // 2
    # halves search space each time
```

Space complexity — what to count: - Input space (usually excluded unless asked for auxiliary space) - Call stack depth for recursion - Extra data structures allocated - **Auxiliary space** = space excluding input

```python
# O(1) space — no extra allocation
def find_max(arr):
    m = arr[0]
    for x in arr:
        m = max(m, x)
    return m

# O(n) space — stores extra array
def prefix_sum(arr):
    ps = [0] * (len(arr) + 1)  # O(n)
    for i, x in enumerate(arr):
        ps[i+1] = ps[i] + x
    return ps

# O(n) space — call stack for recursion
def factorial(n):
    if n <= 1: return 1
    return n * factorial(n - 1)  # n frames on stack
```

Recursion Tree & Master Theorem

Recursion tree method: Draw the tree of recursive calls, compute cost at each level, sum all levels.

DIAGRAM

```
T(n) = 2T(n/2) + n    (merge sort)

Level 0:  n                    → cost n
Level 1:  n/2 + n/2            → cost n
Level 2:  n/4 + n/4 + n/4 + n/4 → cost n
...
Level log n: n leaves of size 1 → cost n

Total: n × (log n + 1) = O(n log n)
```

Master Theorem: For recurrences of the form T(n) = a·T(n/b) + f(n) where a ≥ 1, b > 1:

Let critical exponent: **c* = log_b(a)**

Case	Condition	Result	Intuition
Case 1	f(n) = O(n^(c*−ε)) for some ε > 0	T(n) = Θ(n^c*)	Leaves dominate
Case 2	f(n) = Θ(n^c* · log^k n), k ≥ 0	T(n) = Θ(n^c* · log^(k+1) n)	Equal work per level
Case 3	f(n) = Ω(n^(c*+ε)) and regularity holds	T(n) = Θ(f(n))	Root dominates

Master Theorem examples:

Recurrence	a	b	c*	f(n)	Case	Result
T(n) = 2T(n/2) + n	2	2	1	n = Θ(n¹)	2	Θ(n log n)
T(n) = 2T(n/2) + 1	2	2	1	1 = O(n^0.5)	1	Θ(n)
T(n) = 4T(n/2) + n²	4	2	2	n² = Θ(n²)	2	Θ(n² log n)
T(n) = 4T(n/2) + n³	4	2	2	n³ = Ω(n^2.5)	3	Θ(n³)
T(n) = T(n/2) + 1	1	2	0	1 = Θ(1)	2	Θ(log n)
T(n) = 3T(n/3) + n	3	3	1	n = Θ(n¹)	2	Θ(n log n)

Master Theorem does NOT apply when: - Not in form a·T(n/b) + f(n) — e.g., T(n) = T(n−1) + n (linear decrease, not fraction) - T(n) = T(n/2) + T(n/3) + n (unequal subproblems) - For T(n) = T(n−1) + n → sum 1+2+...+n = O(n²) - For T(n) = T(n−1) + 1 → n recursive calls → O(n)

Input Size → Acceptable Complexity

Use this table to instantly identify which algorithms are feasible given constraints.

n (input size)	Max acceptable complexity	Typical algorithm
n ≤ 10	O(n!) or O(n^n)	Brute-force, permutations
n ≤ 20	$O(2^n \cdot n)$	Bitmask DP, meet in the middle
n ≤ 100	$O(n^3)$ or $O(n^4)$	Floyd-Warshall, naive string matching
n ≤ 1,000	$O(n^2)$	Insertion sort, naive DP
n ≤ 10,000	$O(n^2 \log n)$	Slower $O(n^2)$ algorithms
n ≤ 100,000	O(n log n) or $O(n\sqrt{n})$	Merge sort, segment tree, sqrt decomp
n ≤ 1,000,000	O(n log n)	Merge sort, heap operations
n ≤ 10,000,000	O(n)	Linear scans, counting sort
n > 10^8	O(log n) or O(1)	Binary search, math formulas

Rule of thumb: Modern computers do ~10^8 simple operations/second. Design accordingly.

SECTION 03

03 Algorithms

Sorting

Comparison-Based Sorts

Comparison sorts have a theoretical lower bound of **Ω(n log n)** — you cannot do better with only comparison operations.

Merge Sort

PYTHON

```python
def merge_sort(arr):
    if len(arr) ⩽ 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] ⩽ right[j]:   # ⩽ preserves stability
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Key insight: Divide conquer and merge. Recurrence T(n) = 2T(n/2) + n → **O(n log n)** always. **Stability:** Yes — equal elements keep original relative order. **Drawback:** O(n) extra space; cache-unfriendly for large arrays.

Quick Sort

PYTHON

```python
import random

def quicksort(arr, lo, hi):
    if lo < hi:
        p = partition(arr, lo, hi)
        quicksort(arr, lo, p - 1)
        quicksort(arr, p + 1, hi)

def partition(arr, lo, hi):
    # Lomuto partition scheme
    # Randomize pivot to avoid worst-case O(n²) on sorted input
    rand_idx = random.randint(lo, hi)
    arr[rand_idx], arr[hi] = arr[hi], arr[rand_idx]
    pivot = arr[hi]
    i = lo - 1
    for j in range(lo, hi):
        if arr[j] ⩽ pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[hi] = arr[hi], arr[i + 1]
    return i + 1

# Hoare partition (more efficient in practice):
def hoare_partition(arr, lo, hi):
    pivot = arr[(lo + hi) // 2]
    i, j = lo - 1, hi + 1
    while True:
        i += 1
        while arr[i] < pivot: i += 1
        j -= 1
        while arr[j] > pivot: j -= 1
        if i ⩾ j: return j
        arr[i], arr[j] = arr[j], arr[i]
```

Key insight: Pick pivot, partition so left ≤ pivot ≤ right, recurse on both sides. **Pivot strategies:** - Last element: O(n²) on sorted/reverse-sorted input - Random pivot: Expected O(n log n), avoids adversarial inputs - Median-of-three: Good heuristic - **3-way partition (Dutch National Flag):** Best for arrays with many duplicates — partitions into `< pivot`, `== pivot`, `> pivot`

Stability: No — partition swaps break relative order.

Heap Sort

```python
def heap_sort(arr):
    n = len(arr)
    # Build max-heap (heapify from last internal node)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    # Extract elements
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]  # move max to end
        heapify(arr, i, 0)

def heapify(arr, n, i):
    largest = i
    left, right = 2*i + 1, 2*i + 2
    if left < n and arr[left] > arr[largest]: largest = left
    if right < n and arr[right] > arr[largest]: largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
```

Key insight: Build max-heap in O(n), then repeatedly extract-max in O(log n) each. **Stability:** No. **Space:** O(1) in-place (advantage over merge sort).

Insertion Sort

```python
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
```

Key insight: Build sorted portion left-to-right by inserting each element in its correct position. **Best case:** O(n) when already sorted (inner loop never executes). **When to use:** Small arrays (n < 20), nearly-sorted data, online algorithms (elements arrive one by one). **Stability:** Yes.

Non-Comparison Sorts

Non-comparison sorts break the O(n log n) barrier by exploiting structure of keys.

Counting Sort

```python
def counting_sort(arr, max_val):
    count = [0] * (max_val + 1)
    for x in arr:
        count[x] += 1
    # Build cumulative counts for stability
    for i in range(1, max_val + 1):
        count[i] += count[i - 1]
    output = [0] * len(arr)
    for x in reversed(arr):     # reverse for stability
        output[count[x] - 1] = x
        count[x] -= 1
    return output
```

Complexity: O(n + k) time, O(n + k) space where k = range of values. **When to use:** Keys are integers in a known small range. Useless if k >> n.

Radix Sort

```python
def radix_sort(arr):
    if not arr: return arr
    max_val = max(arr)
    exp = 1
    while max_val // exp > 0:
        arr = counting_sort_by_digit(arr, exp)
        exp *= 10
    return arr

def counting_sort_by_digit(arr, exp):
    n = len(arr)
    output = [0] * n
    count = [0] * 10
    for x in arr:
        count[(x // exp) % 10] += 1
    for i in range(1, 10):
        count[i] += count[i - 1]
    for x in reversed(arr):
        idx = (x // exp) % 10
        output[count[idx] - 1] = x
        count[idx] -= 1
    return output
```

Complexity: O(d·(n + k)) where d = digits, k = base (10 for decimal). Effectively O(n) for fixed-width integers. **Stability:** Requires stable inner sort (counting sort). Must process digits from **least significant** to most significant (LSD radix sort).

Bucket Sort

```python
def bucket_sort(arr, num_buckets=10):
    if not arr: return arr
    min_val, max_val = min(arr), max(arr)
    bucket_range = (max_val - min_val) / num_buckets + 1e-9
    buckets = [[] for _ in range(num_buckets)]
    for x in arr:
        idx = int((x - min_val) / bucket_range)
        idx = min(idx, num_buckets - 1)
        buckets[idx].append(x)
    result = []
    for b in buckets:
        b.sort()        # insertion sort works well for small buckets
        result.extend(b)
    return result
```

Complexity: O(n + k) average when data is uniformly distributed; O(n^2) worst case if all elements land in one bucket. **When to use:** Floating-point numbers in a known range, uniformly distributed data.

Sorting Comparison Table

Algorithm	Best	Average	Worst	Space	Stable	Notes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Guaranteed; good for linked lists
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)*	No	Fastest in practice; randomize pivot
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	Cache-unfriendly
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Best for small/nearly-sorted
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	Never use in interviews
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Never use in interviews
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(n+k)	Yes	Only integers in small range
Radix Sort	O(d·n)	O(d·n)	O(d·n)	O(n+k)	Yes	Fixed-width keys
Bucket Sort	O(n+k)	O(n+k)	$O(n^2)$	O(n+k)	Yes	Uniform float data
Tim Sort	O(n)	O(n log n)	O(n log n)	O(n)	Yes	Python's built-in; hybrid merge+insertion

*Quick sort call stack: O(log n) average, O(n) worst.

Stability matters when: You sort by multiple keys sequentially, or the original order of equal elements has meaning (e.g., sort students by grade then by name).

When to use which: - **General purpose:** Quick sort (in-place, fast cache) or merge sort (stable) - **Guaranteed worst case:** Merge sort or heap sort - **Memory-constrained:** Heap sort (O(1) space) - **Nearly sorted:** Insertion sort or Tim sort - **Integer keys, small range:** Counting sort - **Large integers, many of same size:** Radix sort - **Floating point, uniform:** Bucket sort - **Linked lists:** Merge sort (no random access needed)

Searching

Linear Search

```python
def linear_search(arr, target):
    for i, x in enumerate(arr):
        if x == target:
            return i
    return -1
```

O(n) time, O(1) space. Use only when array is unsorted or n is tiny.

Binary Search

Standard binary search:

```python
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2   # avoids integer overflow
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

First occurrence of target:

```python
def first_occurrence(arr, target):
    lo, hi = 0, len(arr) - 1
    result = -1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            result = mid
            hi = mid - 1    # keep searching left
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return result
```

Last occurrence of target:

```python
def last_occurrence(arr, target):
    lo, hi = 0, len(arr) - 1
    result = -1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            result = mid
            lo = mid + 1    # keep searching right
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return result
```

Lower bound (first index where arr[i] >= target):

```python
import bisect

def lower_bound(arr, target):
    return bisect.bisect_left(arr, target)

# Manual implementation:
def lower_bound_manual(arr, target):
    lo, hi = 0, len(arr)   # hi = len(arr), not len-1
    while lo < hi:          # lo < hi, not lo <= hi
        mid = (lo + hi) // 2
        if arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid        # not mid-1
    return lo
```

Binary search on rotated sorted array:

```python
def search_rotated(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo ⩽ hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        # Left half is sorted
        if arr[lo] ⩽ arr[mid]:
            if arr[lo] ⩽ target < arr[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        # Right half is sorted
        else:
            if arr[mid] < target ⩽ arr[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

Binary search on answer space (parametric search):

The most powerful binary search pattern: instead of searching an array, search for the **answer itself**.

```python
# Example: Koko eating bananas — find minimum eating speed
def min_eating_speed(piles, h):
    def can_finish(speed):
        return sum((p + speed - 1) // speed for p in piles) ⩽ h

    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        if can_finish(mid):
            hi = mid        # mid works, try smaller
        else:
            lo = mid + 1    # too slow, need faster
    return lo

# Template: "find minimum X such that condition(X) is True"
# lo = min possible, hi = max possible
# condition is monotone: False...False...True...True
```

Binary search complexity: O(log n) time, O(1) space.

Binary search mental model: Maintain an invariant — the answer is always in [lo, hi]. Shrink the range by half each iteration. When lo == hi, that's the answer.

Graph Algorithms

Graphs: G = (V, E) where V = vertices (nodes), E = edges.

Representations: - **Adjacency list:** `graph[u] = [(v, weight), ...]` — O(V + E) space, efficient for sparse graphs - **Adjacency matrix:** `graph[u][v] = weight` — O(V^2) space, O(1) edge lookup, efficient for dense graphs

BFS (Breadth-First Search)

```python
from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order

# BFS shortest path (unweighted graph):
def bfs_shortest_path(graph, start, end):
    if start == end: return [start]
    visited = {start: None}   # node → parent
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited[neighbor] = node
                if neighbor == end:
                    path = []
                    cur = end
                    while cur is not None:
                        path.append(cur)
                        cur = visited[cur]
                    return path[::-1]
                queue.append(neighbor)
    return []   # no path
```

Complexity: O(V + E) **Use when:** Shortest path in unweighted graph, level-order traversal, connected components, bipartite check. **Key property:** Visits nodes in order of distance from source — guarantees shortest path in unweighted graphs.

DFS (Depth-First Search)

```python
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    order = []
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                stack.append(neighbor)
    return order

def dfs_recursive(graph, node, visited=None):
    if visited is None: visited = set()
    visited.add(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited)
    return visited
```

Complexity: O(V + E) **Use when:** Cycle detection, topological sort, connected components, path existence, flood fill. **Key property:** Explores as deep as possible before backtracking.

Topological Sort

Valid only for **Directed Acyclic Graphs (DAGs)**.

Kahn's Algorithm (BFS-based):

```python
from collections import deque

def topological_sort_kahn(graph, n):
    """
    graph: adjacency list {node: [neighbors]}
    n: number of nodes (0 to n-1)
    Returns topological order, or empty list if cycle exists.
    """
    in_degree = [0] * n
    for u in range(n):
        for v in graph[u]:
            in_degree[v] += 1

    queue = deque(u for u in range(n) if in_degree[u] == 0)
    order = []

    while queue:
        u = queue.popleft()
        order.append(u)
        for v in graph[u]:
            in_degree[v] -= 1
            if in_degree[v] == 0:
                queue.append(v)

    if len(order) != n:
        return []  # cycle detected — not a DAG
    return order
```

DFS-based topological sort:

```python
def topological_sort_dfs(graph, n):
    WHITE, GRAY, BLACK = 0, 1, 2
    color = [WHITE] * n
    order = []
    has_cycle = [False]

    def dfs(u):
        if has_cycle[0]: return
        color[u] = GRAY
        for v in graph[u]:
            if color[v] == GRAY:
                has_cycle[0] = True
                return
            if color[v] == WHITE:
                dfs(v)
        color[u] = BLACK
        order.append(u)   # add to order AFTER all descendants

    for u in range(n):
        if color[u] == WHITE:
            dfs(u)

    if has_cycle[0]:
        return []
    return order[::-1]    # reverse post-order = topological order
```

Complexity: O(V + E) for both methods. **Kahn vs DFS:** - Kahn: Intuitive, easy cycle detection (len(order) != n), produces lexicographically smallest order with a min-heap - DFS: More natural for recursive thinking, easier to add finish-time ordering

Use cases: Build systems (Makefile), course prerequisites, task scheduling, dependency resolution.

Dijkstra's Algorithm

Shortest path from single source; **requires non-negative edge weights**.

PYTHON

```python
import heapq

def dijkstra(graph, start, n):
    """
    graph: adjacency list {u: [(v, weight), ...]}
    Returns dist[] where dist[v] = shortest distance from start to v.
    """
    dist = [float('inf')] * n
    dist[start] = 0
    # Min-heap: (distance, node)
    pq = [(0, start)]

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue    # stale entry — already found shorter path
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))

    return dist

# With path reconstruction:
def dijkstra_with_path(graph, start, end, n):
    dist = [float('inf')] * n
    prev = [-1] * n
    dist[start] = 0
    pq = [(0, start)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue
        if u == end: break
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                heapq.heappush(pq, (dist[v], v))
    # Reconstruct path
    path = []
    cur = end
    while cur != -1:
        path.append(cur)
        cur = prev[cur]
    return dist[end], path[::-1]
```

Complexity: O((V + E) log V) with binary heap, O(V^2 + E) with array (dense graphs). **When to use:** Single-source shortest path, non-negative weights. **Does NOT work with negative edges** — use Bellman-Ford instead.

Bellman-Ford Algorithm

Shortest path from single source; **handles negative edge weights**; detects negative cycles.

PYTHON

```python
def bellman_ford(edges, n, start):
    """
    edges: list of (u, v, weight)
    Returns dist[] or None if negative cycle reachable from start.
    """
    dist = [float('inf')] * n
    dist[start] = 0

    # Relax all edges V-1 times
    for _ in range(n - 1):
        updated = False
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                updated = True
        if not updated:
            break   # converged early

    # Check for negative cycles (V-th relaxation)
    for u, v, w in edges:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            return None  # negative cycle detected

    return dist
```

Complexity: O(V·E) **When to use:** Negative edge weights, detecting negative cycles (e.g., currency arbitrage). **Guaranteed correct after V−1 iterations** because any shortest path visits at most V−1 edges (in a graph with no negative cycles).

Floyd-Warshall Algorithm

All-pairs shortest paths; handles negative edges; detects negative cycles.

PYTHON

```python
def floyd_warshall(n, edges):
    """Returns dist[i][j] = shortest path from i to j."""
    INF = float('inf')
    dist = [[INF] * n for _ in range(n)]
    for i in range(n):
        dist[i][i] = 0
    for u, v, w in edges:
        dist[u][v] = min(dist[u][v], w)   # handle multi-edges

    for k in range(n):          # intermediate vertex
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    # Check negative cycles: dist[i][i] < 0 for some i
    for i in range(n):
        if dist[i][i] < 0:
            return None  # negative cycle

    return dist
```

Complexity: $O(V^3)$ time, $O(V^2)$ space. **When to use:** All-pairs shortest paths, transitive closure, detecting negative cycles.

Graph Algorithm Comparison Table

Algorithm	Handles Negative Edges	Handles Negative Cycles	Single/All Pairs	Complexity	Notes
Dijkstra	No	No	Single source	O((V+E) log V)	Fastest for non-negative; use with priority queue
Bellman-Ford	Yes	Detects	Single source	O(V·E)	Use when negative edges exist
Floyd-Warshall	Yes	Detects	All pairs	$O(V^3)$	Best for dense graphs, all-pairs
BFS	No (unit weights only)	No	Single source	O(V+E)	Optimal for unweighted graphs
DAG Shortest Path	Yes (DAG only)	N/A	Single source	O(V+E)	Topo sort + relax; fastest for DAGs

Minimum Spanning Tree (MST)

MST of a connected weighted undirected graph: a spanning tree with minimum total edge weight.

Kruskal's Algorithm:

PYTHON

```python
def kruskal(n, edges):
    """
    edges: list of (weight, u, v)
    Returns MST weight and edges.
    """
    edges.sort()   # sort by weight
    parent = list(range(n))
    rank = [0] * n

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]  # path compression (2-step)
            x = parent[x]
        return x

    def union(x, y):
        px, py = find(x), find(y)
        if px == py: return False   # same component — cycle
        if rank[px] < rank[py]:
            px, py = py, px
        parent[py] = px
        if rank[px] == rank[py]:
            rank[px] += 1
        return True

    mst_weight = 0
    mst_edges = []
    for w, u, v in edges:
        if union(u, v):
            mst_weight += w
            mst_edges.append((u, v, w))
            if len(mst_edges) == n - 1:
                break

    return mst_weight, mst_edges
```

Prim's Algorithm:

```python
import heapq

def prim(graph, n):
    """
    graph: adjacency list {u: [(v, weight), ...]}
    Returns MST weight.
    """
    visited = set()
    # Start from node 0: (weight, node)
    pq = [(0, 0)]
    mst_weight = 0

    while pq and len(visited) < n:
        w, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)
        mst_weight += w
        for v, weight in graph[u]:
            if v not in visited:
                heapq.heappush(pq, (weight, v))

    return mst_weight if len(visited) == n else -1  # -1 if disconnected
```

Kruskal vs Prim: | | Kruskal | Prim | |-|---------|------| | Approach | Edge-centric: sort all edges, add if no cycle | Vertex-centric: grow tree from one vertex | | Complexity | O(E log E) | O((V+E) log V) with heap | | Best for | Sparse graphs | Dense graphs | | Data structure | Union-Find | Priority queue |

Union-Find (Disjoint Set Union)

```python
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.components = n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])   # path compression
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py: return False
        # Union by rank
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1
        self.components -= 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)
```

Amortized complexity: O(α(n)) per operation where α is inverse Ackermann — effectively O(1). **Use for:** Cycle detection, Kruskal's MST, connected components, dynamic connectivity.

Cycle Detection

Undirected graph — Union-Find:

```python
def has_cycle_undirected(n, edges):
    uf = UnionFind(n)
    for u, v in edges:
        if not uf.union(u, v):
            return True   # u and v already in same component
    return False
```

Undirected graph — DFS (coloring):

```python
def has_cycle_undirected_dfs(graph, n):
    visited = set()
    def dfs(node, parent):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                if dfs(neighbor, node): return True
            elif neighbor != parent:    # back edge to non-parent = cycle
                return True
        return False
    for i in range(n):
        if i not in visited:
            if dfs(i, -1): return True
    return False
```

Directed graph — DFS with 3-color:

```python
def has_cycle_directed(graph, n):
    WHITE, GRAY, BLACK = 0, 1, 2
    color = [WHITE] * n
    def dfs(u):
        color[u] = GRAY
        for v in graph[u]:
            if color[v] == GRAY: return True    # back edge = cycle
            if color[v] == WHITE and dfs(v): return True
        color[u] = BLACK
        return False
    return any(dfs(u) for u in range(n) if color[u] == WHITE)
```

Key difference: In undirected graphs, visiting a neighbor (other than parent) that's already visited = cycle. In directed graphs, visiting a GRAY (currently-being-processed) node = cycle (back edge).

Connected Components

```python
def connected_components(graph, n):
    visited = set()
    components = []
    def dfs(node, component):
        visited.add(node)
        component.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                dfs(neighbor, component)
    for i in range(n):
        if i not in visited:
            component = []
            dfs(i, component)
            components.append(component)
    return components
```

Bipartite Check

```python
from collections import deque

def is_bipartite(graph, n):
    color = [-1] * n
    for start in range(n):
        if color[start] != -1: continue
        color[start] = 0
        queue = deque([start])
        while queue:
            u = queue.popleft()
            for v in graph[u]:
                if color[v] == -1:
                    color[v] = 1 - color[u]
                    queue.append(v)
                elif color[v] == color[u]:
                    return False    # same color = odd cycle = not bipartite
    return True
```

A graph is bipartite iff it has no odd-length cycles. Two-color with BFS; if conflict found, not bipartite.

Dynamic Programming

DP solves problems by **breaking them into overlapping subproblems** and storing results to avoid recomputation.

Memoization vs Tabulation

	Memoization (Top-Down)	Tabulation (Bottom-Up)
Approach	Recursive with cache	Iterative, fill table
Direction	Problem → subproblems	Base cases → problem
Space	O(depth) stack + memo	O(states) table
Complexity	Same asymptotically	Often faster constants
Ease	Easier to implement	Better for optimization
When to use	Not all subproblems needed	All subproblems computed

```python
# Memoization (Fibonacci):
from functools import lru_cache

@lru_cache(maxsize=None)
def fib_memo(n):
    if n <= 1: return n
    return fib_memo(n-1) + fib_memo(n-2)

# Tabulation (Fibonacci):
def fib_tab(n):
    if n <= 1: return n
    dp = [0, 1]
    for i in range(2, n+1):
        dp.append(dp[-1] + dp[-2])
    return dp[n]

# Space-optimized (only need last 2):
def fib_opt(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

How to Identify DP Problems

DP applies when: 1. Problem asks for **optimal value** (min/max) or **count of ways** 2. Subproblems **overlap** (same subproblem solved repeatedly) 3. Problem has **optimal substructure** (optimal solution uses optimal solutions to subproblems) 4. Problem asks: "is it possible to achieve X?"

DP does NOT apply when: - No overlapping subproblems (use divide and conquer) - Greedy works (exchange argument) - Subproblems are independent

DP design recipe: 1. Define the **state** — what information uniquely identifies a subproblem? 2. Write the **recurrence** (transition function) 3. Identify **base cases** 4. Determine **computation order** (which states depend on which) 5. Answer — which state(s) give the final answer?

Classic DP Families

0/1 Knapsack: Items with weight and value; capacity W; each item used at most once. Recurrence: `dp[i][w] = max(dp[i-1][w], dp[i-1][w-weight[i]] + value[i])`

```python
def knapsack_01(weights, values, W):
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(W + 1):
            dp[i][w] = dp[i-1][w]   # don't take item i
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w], dp[i-1][w - weights[i-1]] + values[i-1])
    return dp[n][W]

# Space-optimized (1D DP) — iterate w in reverse:
def knapsack_01_opt(weights, values, W):
    dp = [0] * (W + 1)
    for i in range(len(weights)):
        for w in range(W, weights[i] - 1, -1):   # MUST go right to left
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
    return dp[W]
```

Unbounded Knapsack (items can be reused):

```python
def knapsack_unbounded(weights, values, W):
    dp = [0] * (W + 1)
    for w in range(W + 1):
        for i in range(len(weights)):
            if weights[i] <= w:
                dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
    return dp[W]
```

Coin Change (minimum coins): Recurrence: `dp[amount] = min(dp[amount - coin] + 1) for each coin`

```python
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for coin in coins:
            if coin <= a:
                dp[a] = min(dp[a], dp[a - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

Coin Change (number of ways):

```python
def coin_change_ways(coins, amount):
    dp = [0] * (amount + 1)
    dp[0] = 1
    for coin in coins:          # order matters: outer=coins for combinations
        for a in range(coin, amount + 1):
            dp[a] += dp[a - coin]
    return dp[amount]
```

Longest Common Subsequence (LCS): Recurrence: - If s1[i] == s2[j]: `dp[i][j] = dp[i-1][j-1] + 1` - Else: `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`

```python
def lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

Edit Distance (Levenshtein): Operations: insert, delete, replace (each cost 1). Recurrence: - If s1[i] == s2[j]: `dp[i][j] = dp[i-1][j-1]` - Else: `dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])`

```python
def edit_distance(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1): dp[i][0] = i
    for j in range(n + 1): dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j],     # delete from s1
                                   dp[i][j-1],      # insert into s1
                                   dp[i-1][j-1])    # replace
    return dp[m][n]
```

Longest Increasing Subsequence (LIS):

```python
# O(n²) DP:
def lis_n2(arr):
    n = len(arr)
    dp = [1] * n
    for i in range(1, n):
        for j in range(i):
            if arr[j] < arr[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

# O(n log n) patience sorting:
import bisect

def lis_nlogn(arr):
    tails = []  # tails[i] = smallest tail element of all IS of length i+1
    for x in arr:
        pos = bisect.bisect_left(tails, x)
        if pos == len(tails):
            tails.append(x)
        else:
            tails[pos] = x
    return len(tails)
```

Matrix Chain Multiplication (Interval DP): Recurrence: `dp[i][j] = min over k of (dp[i][k] + dp[k+1][j] + dims[i-1]*dims[k]*dims[j])`

```python
def matrix_chain(dims):
    n = len(dims) - 1   # number of matrices
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n + 1):      # subproblem length
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float('inf')
            for k in range(i, j):
                cost = dp[i][k] + dp[k+1][j] + dims[i] * dims[k+1] * dims[j+1]
                dp[i][j] = min(dp[i][j], cost)
    return dp[0][n-1]
```

Matrix Paths (unique paths, min cost path):

```python
def unique_paths(m, n):
    dp = [[1] * n for _ in range(m)]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]
    return dp[m-1][n-1]

def min_path_sum(grid):
    m, n = len(grid), len(grid[0])
    dp = [[0] * n for _ in range(m)]
    dp[0][0] = grid[0][0]
    for i in range(1, m): dp[i][0] = dp[i-1][0] + grid[i][0]
    for j in range(1, n): dp[0][j] = dp[0][j-1] + grid[0][j]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])
    return dp[m-1][n-1]
```

DP on Trees:

```python
def tree_dp_max_independent_set(adj, n):
    # dp[node][0] = max IS if node NOT included
    # dp[node][1] = max IS if node included
    dp = [[0, 1] for _ in range(n)]   # [0] = not taken, [1] = taken (value 1)

    def dfs(u, parent):
        for v in adj[u]:
            if v == parent: continue
            dfs(v, u)
            # If u not taken, v can be taken or not
            dp[u][0] += max(dp[v][0], dp[v][1])
            # If u taken, v cannot be taken
            dp[u][1] += dp[v][0]

    dfs(0, -1)
    return max(dp[0][0], dp[0][1])
```

Bitmask DP (Travelling Salesman): State: `dp[mask][i]` = minimum cost to visit all cities in `mask`, ending at city `i`.

```python
def tsp(dist, n):
    INF = float('inf')
    dp = [[INF] * n for _ in range(1 << n)]
    dp[1][0] = 0   # visited city 0, at city 0

    for mask in range(1, 1 << n):
        for u in range(n):
            if not (mask >> u & 1): continue
            if dp[mask][u] == INF: continue
            for v in range(n):
                if mask >> v & 1: continue   # already visited
                new_mask = mask | (1 << v)
                dp[new_mask][v] = min(dp[new_mask][v], dp[mask][u] + dist[u][v])

    full = (1 << n) - 1
    return min(dp[full][v] + dist[v][0] for v in range(n))
```

DP Key Takeaways: - **State is everything** — spend most time defining what uniquely describes a subproblem - **Recurrence is mechanical** — once state is defined, transitions follow naturally - **Base cases are critical** — wrong base cases cause wrong answers even with correct recurrence - **Space optimization:** 2D DP can often become 1D; 1D can become O(1) if only adjacent states needed

Greedy Algorithms

Greedy algorithms make the **locally optimal choice at each step** hoping to reach a global optimum.

When Greedy Works

Greedy is correct when two conditions hold: 1. **Greedy choice property:** A globally optimal solution can always be reached by making locally optimal choices 2. **Optimal substructure:** Optimal solution contains optimal solutions to subproblems

Exchange argument: Standard proof technique. Assume greedy is not optimal. Show that any optimal solution can be transformed (by swapping) to match greedy without increasing cost — contradiction.

Classic Greedy Problems

Activity Selection (Interval Scheduling): Select maximum number of non-overlapping intervals. Strategy: Sort by end time; always pick the activity that finishes earliest.

```python
def activity_selection(intervals):
    intervals.sort(key=lambda x: x[1])   # sort by end time
    count = 0
    last_end = float('-inf')
    for start, end in intervals:
        if start ⩾ last_end:
            count += 1
            last_end = end
    return count
```

Huffman Coding: Build optimal prefix-free codes using a min-heap. Greedy: always merge the two lowest-frequency nodes.

Fractional Knapsack: (unlike 0/1 knapsack, greedy works here)

```python
def fractional_knapsack(weights, values, W):
    ratios = sorted(zip(values, weights), key=lambda x: x[0]/x[1], reverse=True)
    total = 0
    for v, w in ratios:
        if W ⩾ w:
            total += v; W -= w
        else:
            total += v * W / w; break
    return total
```

Jump Game: Can you reach the end of an array where each element is max jump length?

```python
def can_jump(nums):
    max_reach = 0
    for i, jump in enumerate(nums):
        if i > max_reach: return False
        max_reach = max(max_reach, i + jump)
    return True
```

Task Scheduling: Schedule tasks with deadlines to maximize profit. Strategy: Sort by profit descending; greedily assign tasks to latest available slot.

Gas Station: Find starting station to complete circular tour.

```python
def can_complete_circuit(gas, cost):
    total_surplus = 0
    surplus = 0
    start = 0
    for i in range(len(gas)):
        diff = gas[i] - cost[i]
        total_surplus += diff
        surplus += diff
        if surplus < 0:
            start = i + 1
            surplus = 0
    return start if total_surplus >= 0 else -1
```

Greedy vs DP: Greedy is O(n log n) or O(n); DP is O(n^2) or worse. **Prefer greedy when provably correct** (exchange argument or known greedy structure). DP is the safe fallback.

Recursion & Backtracking

Recursion Template

```python
def solve(state, ...):
    # Base case
    if is_complete(state):
        record_solution(state)
        return

    # Recursive case
    for choice in get_choices(state):
        if is_valid(choice, state):
            make_choice(state, choice)
            solve(state, ...)
            undo_choice(state, choice)   # backtrack
```

Backtracking Framework

Backtracking = DFS on the decision tree with **pruning** to avoid exploring invalid paths.

```python
# Subsets (power set):
def subsets(nums):
    result = []
    def backtrack(start, path):
        result.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()
    backtrack(0, [])
    return result

# Permutations:
def permutations(nums):
    result = []
    def backtrack(path, used):
        if len(path) == len(nums):
            result.append(path[:])
            return
        for i in range(len(nums)):
            if used[i]: continue
            used[i] = True
            path.append(nums[i])
            backtrack(path, used)
            path.pop()
            used[i] = False
    backtrack([], [False] * len(nums))
    return result

# N-Queens:
def solve_n_queens(n):
    result = []
    cols = set(); diag1 = set(); diag2 = set()
    board = [['.' for _ in range(n)] for _ in range(n)]
    def backtrack(row):
        if row == n:
            result.append([''.join(r) for r in board])
            return
        for col in range(n):
            if col in cols or (row-col) in diag1 or (row+col) in diag2:
                continue
            cols.add(col); diag1.add(row-col); diag2.add(row+col)
            board[row][col] = 'Q'
            backtrack(row + 1)
            cols.remove(col); diag1.remove(row-col); diag2.remove(row+col)
            board[row][col] = '.'
    backtrack(0)
    return result
```

Pruning strategies: - **Constraint propagation:** Skip branches that violate constraints immediately - **Bound pruning:** Skip branches that cannot improve on best known solution - **Symmetry breaking:** Avoid exploring symmetrically equivalent states - **Early termination:** Return as soon as one solution found (if only need one)

Backtracking complexity: Exponential in worst case (O(n!), $O(2^n)$), but pruning can dramatically reduce actual runtime.

Backtracking is NOT DP: Backtracking explores all possibilities with pruning; DP stores subproblem results to avoid recomputation. Backtracking problems don't have overlapping subproblems.

Bit Manipulation

Computers operate on binary; bit tricks exploit this for O(1) operations.

Core Operations

Operation	Expression	Effect
Set bit k	`n |= (1 << k)`	Force bit k to 1
Clear bit k	`n &= ~(1 << k)`	Force bit k to 0
Toggle bit k	`n ^= (1 << k)`	Flip bit k
Check bit k	`(n >> k) & 1`	Get value of bit k
Clear lowest set bit	`n & (n - 1)`	Removes rightmost 1 bit
Isolate lowest set bit	`n & (-n)`	Gets rightmost 1 bit only
Check power of 2	`n > 0 and (n & (n-1)) == 0`	True iff exactly one bit set
Count set bits	`bin(n).count('1')` or `n.bit_count()`	Hamming weight

Key XOR Tricks

PYTHON

```python
# XOR properties:
# a ^ a = 0 (anything XOR itself = 0)
# a ^ 0 = a (anything XOR 0 = itself)
# XOR is commutative and associative

# Find single number in array where all others appear twice:
def single_number(nums):
    result = 0
    for x in nums:
        result ^= x   # pairs cancel out
    return result

# Find two unique numbers when all others appear twice:
def single_number_two(nums):
    xor = 0
    for x in nums: xor ^= x   # xor of the two unique numbers
    # Find any set bit in xor (they differ in this bit)
    diff_bit = xor & (-xor)
    a = 0
    for x in nums:
        if x & diff_bit:
            a ^= x   # group by this bit
    return a, xor ^ a

# Swap without temp:
a ^= b; b ^= a; a ^= b

# Check if two integers have opposite signs:
def opposite_signs(a, b):
    return (a ^ b) < 0   # MSB will be 1 if signs differ
```

Bitmask DP

```python
# Enumerate all subsets of a bitmask:
mask = 0b1011   # some bitmask
sub = mask
while sub > 0:
    process(sub)   # sub is a subset of mask
    sub = (sub - 1) & mask   # get next subset

# Check if bit i is set in mask:
if mask & (1 << i):
    pass

# Count set bits (Brian Kernighan's algorithm):
def count_bits(n):
    count = 0
    while n:
        n &= (n - 1)   # remove lowest set bit
        count += 1
    return count

# Generate all 2^n subsets:
n = 4
for mask in range(1 << n):
    subset = [i for i in range(n) if mask & (1 << i)]
```

Bit Tricks Summary

```python
# Multiply/divide by power of 2:
x << k  == x * 2^k
x >> k  == x // 2^k

# Modulo by power of 2:
x & (m - 1) == x % m   # only works when m is power of 2

# Average without overflow:
avg = (a & b) + ((a ^ b) >> 1)

# Absolute value (no branching):
mask = x >> 31   # -1 if negative, 0 if positive
abs_x = (x + mask) ^ mask

# Next power of 2:
def next_power_of_2(n):
    n -= 1
    n |= n >> 1; n |= n >> 2; n |= n >> 4
    n |= n >> 8; n |= n >> 16
    return n + 1
```

Number Theory / Math

GCD and LCM

PYTHON

```python
import math

# GCD using Euclidean algorithm: gcd(a, b) = gcd(b, a % b)
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

# LCM: lcm(a, b) = a * b / gcd(a, b)
def lcm(a, b):
    return a * b // gcd(a, b)

# Python 3.9+:
math.gcd(a, b)
math.lcm(a, b)

# GCD complexity: O(log(min(a, b))) — Fibonacci numbers are worst case
```

Extended Euclidean Algorithm: Finds x, y such that a·x + b·y = gcd(a, b):

PYTHON

```python
def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x, y = extended_gcd(b, a % b)
    return g, y, x - (a // b) * y
```

Sieve of Eratosthenes

Find all primes up to n in O(n log log n):

PYTHON

```python
def sieve(n):
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] = False
    for i in range(2, int(n**0.5) + 1):
        if is_prime[i]:
            for j in range(i*i, n+1, i):   # start from i² (smaller multiples already marked)
                is_prime[j] = False
    return [i for i in range(n+1) if is_prime[i]]

# Segmented sieve for large ranges:
# Prime factorization using sieve (smallest prime factor):
def smallest_prime_factor(n):
    spf = list(range(n + 1))
    for i in range(2, int(n**0.5) + 1):
        if spf[i] == i:   # i is prime
            for j in range(i*i, n+1, i):
                if spf[j] == j:
                    spf[j] = i
    return spf
```

Modular Arithmetic

For large number problems where answer is `mod 10^9 + 7`:

```python
MOD = 10**9 + 7

# Basic operations:
(a + b) % MOD
(a - b + MOD) % MOD   # add MOD before % to avoid negative
(a * b) % MOD
(a ** b) % MOD == pow(a, b, MOD)   # Python's built-in is efficient

# Modular inverse (when MOD is prime, use Fermat's little theorem):
# a^(MOD-1) ≡ 1 (mod MOD) when MOD is prime
# Therefore a^(-1) ≡ a^(MOD-2) (mod MOD)
def mod_inverse(a, mod):
    return pow(a, mod - 2, mod)   # O(log mod)

# Modular inverse using extended GCD (works for any mod, gcd(a,mod)=1):
def mod_inverse_gcd(a, mod):
    g, x, _ = extended_gcd(a, mod)
    if g != 1: raise ValueError("Inverse doesn't exist")
    return x % mod

# Modular combinations:
def mod_choose(n, k, mod=10**9+7):
    if k > n: return 0
    # Precompute factorials
    fact = [1] * (n + 1)
    for i in range(1, n+1):
        fact[i] = fact[i-1] * i % mod
    inv_fact = [1] * (n + 1)
    inv_fact[n] = pow(fact[n], mod - 2, mod)
    for i in range(n-1, -1, -1):
        inv_fact[i] = inv_fact[i+1] * (i+1) % mod
    return fact[n] * inv_fact[k] % mod * inv_fact[n-k] % mod
```

Fast Exponentiation (Binary Exponentiation)

Compute a^b in O(log b):

```python
def fast_pow(base, exp, mod=None):
    result = 1
    while exp > 0:
        if exp & 1:               # if current bit is 1
            result = result * base
            if mod: result %= mod
        base = base * base
        if mod: base %= mod
        exp >>= 1                 # shift right (divide by 2)
    return result

# Python's built-in pow(a, b, mod) uses this algorithm
```

Key idea: a^13 = a^(1101 in binary) = a^8 · a^4 · a^1. Square the base repeatedly, multiply result when bit is 1.

Prime Factorization

```python
def prime_factors(n):
    factors = {}
    d = 2
    while d * d <= n:
        while n % d == 0:
            factors[d] = factors.get(d, 0) + 1
            n //= d
        d += 1
    if n > 1:
        factors[n] = factors.get(n, 0) + 1
    return factors
# Complexity: O(√n)
```

Useful Math Facts

- Number of divisors of n: O(n^(1/3)) on average
- Sum of 1 to n: n(n+1)/2
- Sum of squares 1 to n: n(n+1)(2n+1)/6
- Catalan numbers: C(2n,n)/(n+1) — counts BSTs, valid parentheses, polygon triangulations
- **Pigeonhole principle:** n+1 items in n containers → at least one container has ≥ 2 items
- **Stars and bars:** Distribute n identical items into k bins: C(n+k-1, k-1)

SECTION 04

04 Algorithm Selection Cheat Table

Problem Type	Key Signal	Recommended Algorithm	Complexity
Shortest path, unweighted	BFS in grid/graph	BFS	O(V+E)
Shortest path, non-negative weights	Edge weights, single source	Dijkstra	O((V+E)log V)
Shortest path, negative weights	Negative edges allowed	Bellman-Ford	O(VE)
All-pairs shortest path	Need dist between all pairs	Floyd-Warshall	$O(V^3)$
Minimum spanning tree	Connect all nodes cheaply	Kruskal or Prim	O(E log E)
Sort, general	n ≤ 10^6	Quick sort / Merge sort	O(n log n)
Sort, small integer range	Keys in [0, k]	Counting sort / Radix sort	O(n+k)
Cycle detection, undirected	Find cycle in graph	Union-Find or DFS	O(α(n)) or O(V+E)
Cycle detection, directed	Find cycle in digraph	DFS (3-color)	O(V+E)
Topological order	DAG dependency order	Kahn's or DFS topo	O(V+E)
Connected components	Group nodes	DFS/BFS + visited set	O(V+E)
Optimal value with choices	Overlapping subproblems	Dynamic Programming	Varies
Make decisions greedily	Exchange argument holds	Greedy	O(n log n) or O(n)
All subsets/permutations	Enumerate possibilities	Backtracking	O(2^n) or O(n!)
Find element in sorted array	Array is sorted	Binary search	O(log n)
Find minimum X satisfying condition	Monotone predicate on integers	Binary search on answer	O(log n · check)
Frequency counting	Count occurrences	Hash map / Counting array	O(n)
Range minimum/maximum	Static range queries	Sparse table	O(n log n) pre, O(1) query
Range sum queries	Prefix sums or updates	Prefix sum / Fenwick tree	O(n) or O(n log n)
Substring search	Pattern in text	KMP / Rabin-Karp	O(n+m)
Disjoint set operations	Dynamic connectivity	Union-Find	O(α(n)) amortized
Large integers, divisibility	Modular arithmetic	Fermat's theorem, sieve	O(log n)
Find unique / pair in XOR	XOR properties	Bit manipulation	O(n)
Multiple choices at each step	Combinatorial search	Backtracking + pruning	Exponential
Maximize non-overlapping intervals	Interval scheduling	Greedy (sort by end time)	O(n log n)

SECTION 05

05 Real-Life Analogies

Algorithm / Concept	Real-Life Analogy
BFS	Ripple in a pond — spreads layer by layer from center
DFS	Solving a maze by always turning left until you hit a wall, then backtracking
Dijkstra	Google Maps routing — always expand the nearest unvisited city
Bellman-Ford	Send rumors through a network; after n-1 rounds, everyone knows the shortest path
Dynamic Programming	Filling out a tax form — use last year's answers to compute this year's
Memoization	Sticky notes — write down each sub-answer once so you never recalculate it
Greedy (interval scheduling)	Booking meeting rooms — always pick the meeting that ends soonest
Merge Sort	Library sorting books by splitting into piles, sorting each pile, then merging
Quick Sort	Separating audience by height relative to a pivot person, then recursing
Binary Search	Guessing a number — always guess the middle of the remaining range
Topological Sort	Course prerequisite ordering — take prerequisites before the main course
Union-Find	Friend groups — if A and B are friends, merge their groups; check if same group
Backtracking	Trying all keys on a keyring — discard a key as soon as it doesn't fit
Sieve of Eratosthenes	Crossing out multiples on a number grid to find primes
Amortized Analysis	Buying coffee with a punch card — most coffees are cheap, the 10th is free
Big-O	Speed limit signs — tells you the worst that can happen, not typical speed
MST (Kruskal)	Building cheapest road network by adding roads from cheapest to most expensive
Hash Table	Lockers with assigned numbers — direct access by computing the locker number
Heap	Hospital triage — the most critical patient always treated next

SECTION 06

06 Common Interview Questions

Complexity Analysis

- What is the time complexity of Python's `list.sort()`? (O(n log n) — Tim Sort)
- What is the amortized complexity of appending to a Python list? (O(1))
- Given a recursive function, derive its time complexity using Master Theorem
- What is the space complexity of BFS vs DFS? (BFS: O(V) for queue; DFS: O(V) for stack/recursion depth)

Sorting

- Sort an array of 0s, 1s, and 2s without using built-in sort (Dutch National Flag — O(n))
- Find the k-th largest element in an array (QuickSelect — O(n) avg, or heap — O(n log k))
- Check if an array is sorted in O(n)
- Merge two sorted arrays in-place

Binary Search

- Find first and last position of element in sorted array (LeetCode 34)
- Search in rotated sorted array (LeetCode 33)
- Find minimum in rotated sorted array (LeetCode 153)
- Koko eating bananas / minimum capacity ship (binary search on answer space)
- Find peak element (LeetCode 162 — binary search on unsorted array!)

Graphs

- Number of islands (DFS/BFS on grid — LeetCode 200)
- Course schedule — detect cycle in directed graph (LeetCode 207)
- Word ladder — BFS shortest path (LeetCode 127)
- Clone graph (DFS with memo)
- Cheapest flights within k stops (modified Dijkstra/Bellman-Ford)
- Reconstruct itinerary (Eulerian path via DFS)
- Network delay time (Dijkstra — LeetCode 743)
- Find if path exists — Union-Find
- Minimum cost to connect all points — Prim's MST (LeetCode 1584)
- Critical connections / bridges (Tarjan's algorithm)

Dynamic Programming

- Climbing stairs / coin change (basic 1D DP)
- Longest common subsequence (classic 2D DP)
- Knapsack variants (0/1, unbounded, multi-dimensional)
- Longest increasing subsequence ($O(n^2)$ DP or O(n log n) patience sort)
- Edit distance / regular expression matching
- Maximum subarray sum (Kadane's algorithm — O(n) DP)
- Maximum product subarray
- House robber / house robber II (circular)
- Unique paths / minimum path sum in grid
- Burst balloons (interval DP — LeetCode 312)
- Word break (1D DP + hash set)
- Partition equal subset sum (subset sum DP)

Backtracking

- N-Queens (LeetCode 51)
- Sudoku solver (LeetCode 37)
- Generate all valid parentheses (LeetCode 22)
- Subsets / permutations / combinations (LeetCode 78, 46, 77)
- Word search in grid (LeetCode 79)

Greedy

- Merge overlapping intervals (LeetCode 56)
- Non-overlapping intervals (LeetCode 435)
- Jump game I and II (LeetCode 55, 45)
- Task scheduler (LeetCode 621)
- Minimum number of arrows to burst balloons (LeetCode 452)

SECTION 07

07 Typical Mistakes Candidates Make

Complexity Analysis Mistakes

- **Forgetting recursion call stack space** — O(n) space for O(n) deep recursion
- **Counting operations instead of asymptotic complexity** — writing "O(2n)" instead of "O(n)"
- **Wrong space complexity for BFS** — BFS can hold O(V) nodes in queue, not O(1)
- **Not accounting for sorting** — saying a solution is O(n) when it first sorts in O(n log n)
- **Misapplying Master Theorem** — forgetting to check regularity condition for Case 3

Graph Algorithm Mistakes

- **Using Dijkstra with negative edges** — it will produce wrong results
- **Not handling disconnected graphs** — BFS/DFS from single source misses other components
- **Off-by-one in Bellman-Ford** — need exactly V-1 iterations, not V
- **Forgetting to check for cycles in topological sort** — not verifying all nodes are processed
- **Confusing directed and undirected cycle detection** — using undirected method on directed graph
- **Not initializing all distances to infinity** — leaving 0 causes wrong Dijkstra results

Sorting Mistakes

- **Using comparison sort when counting/radix would be O(n)**
- **Forgetting stability requirements** — using unstable sort when stability matters
- **Not randomizing quicksort pivot** — gets $O(n^2)$ on sorted input in interviews where test cases include sorted arrays
- **Forgetting in-place constraint** — using merge sort when O(1) space needed

Dynamic Programming Mistakes

- **Defining state incorrectly** — most DP bugs come from wrong state definition
- **Wrong iteration order** — 1D knapsack must iterate weights in reverse for 0/1; forward for unbounded
- **Off-by-one in indices** — dp[i] represents item i or first i items?
- **Not thinking about base cases** — wrong base cases cause cascade of wrong answers
- **Confusing DP with backtracking** — implementing exponential backtracking when DP is needed
- **Forgetting to return correct cell** — computing dp[n][m] when answer is in dp[n-1][m-1]

Binary Search Mistakes

- **Integer overflow with mid = (lo + hi) / 2** — use `lo + (hi - lo) // 2`
- **Infinite loop** — wrong loop condition or mid not shrinking the range
- **Off-by-one in boundary** — `lo < hi` vs `lo <= hi`; `hi = mid` vs `hi = mid - 1`
- **Wrong convergence** — `lower_bound` needs `hi = len(arr)`, not `len(arr) - 1`

General Interview Mistakes

- **Not clarifying constraints** — solution for n=10 differs from n=10^9
- **Not discussing complexity** — implementing a solution without analyzing it
- **Jumping to code** — not spending time designing the algorithm first
- **Not testing with edge cases** — empty input, single element, all duplicates
- **Over-engineering** — writing complex code when a simple $O(n^2)$ is acceptable for given n

SECTION 08

08 Revision Cheat Sheet

Must-Know Complexities

Operation	Data Structure	Time
Access by index	Array	O(1)
Search	Array (unsorted)	O(n)
Search	Array (sorted)	O(log n)
Insert/Delete	Array (arbitrary position)	O(n)
Insert/Delete	Linked List	O(1) at head; O(n) to find
Push/Pop	Stack/Queue	O(1)
Insert/Delete/Search	Hash Table	O(1) avg, O(n) worst
Insert/Delete/Search	BST (balanced)	O(log n)
Insert/Extract-Min	Min-Heap	O(log n)
Build heap from array	Heap	O(n)
DFS/BFS	Graph	O(V+E)
Dijkstra	Graph (non-neg)	O((V+E) log V)
Bellman-Ford	Graph	O(VE)
Floyd-Warshall	Graph	$O(V^3)$
Kruskal/Prim	Graph	O(E log E)
Merge Sort / Heap Sort	Array	O(n log n)
Quick Sort	Array	O(n log n) avg
Counting Sort	Array	O(n+k)
Binary Search	Sorted Array	O(log n)
Union-Find	DSU	O(α(n)) amortized
Sieve of Eratosthenes	-	O(n log log n)
GCD	-	O(log(min(a,b)))
Fast Exponentiation	-	O(log n)

Algorithm Triggers (What to Think When You See...)

You see...	Think...
Shortest path, no weights	BFS
Shortest path, positive weights	Dijkstra
Shortest path, negative weights	Bellman-Ford
All-pairs shortest path	Floyd-Warshall
Minimum spanning tree	Kruskal (sparse) / Prim (dense)
"Connected components" or "groups"	Union-Find or DFS
"Cycle in directed graph"	Topological sort (Kahn's) or DFS 3-color
"Dependency ordering"	Topological sort
"Count ways", "min/max cost"	Dynamic Programming
"Optimal choice at each step"	Greedy (verify with exchange argument)
"All possibilities / enumerate"	Backtracking
"Sorted array, find X"	Binary search
"Find minimum X such that..."	Binary search on answer
"Sliding window, subarray"	Two pointers or sliding window
"Top K elements"	Heap (size K)
"Anagram / frequency"	Hash map or counting array
"Intervals, overlapping"	Sort by start/end + greedy
"Parentheses, matching"	Stack
"Next greater element"	Monotonic stack
"Integer keys, small range"	Counting sort / bucket
"Large integers mod prime"	Modular arithmetic + Fermat
"Find single number in pairs"	XOR trick
"Power of 2 / bit patterns"	Bit manipulation
"String matching"	KMP / Rabin-Karp
"2D grid, path finding"	BFS (shortest) / DFS (any path)
"Subsets, combinations"	Backtracking or bitmask
"Repeated subproblems visible"	Memoize it → DP

Golden Rules

1. **Always clarify constraints before coding** — n determines your algorithm
2. **State complexity before implementation** — shows algorithmic thinking
3. **Draw examples** — small examples reveal patterns
4. **Code the brute force first if stuck** — then optimize
5. **Test edge cases explicitly** — empty, single element, all same, already sorted
6. **For graphs:** always check — directed or undirected? weighted? negative edges? disconnected?
7. **For DP:** define state precisely, write recurrence, then code
8. **For greedy:** justify why local optimal = global optimal
9. **Binary search:** maintain invariant, never drop the answer
10. **Recursion:** trust the recursion — define what it returns, use it correctly

ENGINEERING ESSENTIALS

Master the fundamentals.

Understand the why before the how. Connect every concept to a real system, an analogy, and a trade-off you can defend.

Vol. 14 — DSA Algorithms

FAANG Interview Master Series